

第6章 什么是寄存器

本章参考资料：《STM32F4xx 中文参考手册》、《STM32F4xx 英文数据手册》、
学习本章时，配合《STM32F4xx 中文参考手册》“存储器和总线架构”、“嵌入式
FLASH 接口”及“通用 I/O(GPIO)”章节一起阅读，效果会更佳，特别是涉及到寄存器说
明的部分。

6.1 什么是寄存器

在学习 STM32 编程时，通常有两种编程方式，一种是寄存器编程，另外一种是用库编程，其中寄存器编程是基础，而库编程是在寄存器编程的基础上升级而来的一种易于学习和开发的编程方式，是我们学习 STM32 编程的时候需要重点掌握的一种编程方法。库编程对于项目开发固然简单和快速，但是从学习的角度开发，学习寄存器编程的方法我们也不能丢。其实，我们在学习 8 位或者 16 位单片机的时候，居多都是采用寄存器编程，那么这个寄存器编程里面的寄存器是什么东西？它在芯片的什么地方？在探索 STM32 寄存器深层次的问题时，我们从 STM32 表面的外观开始，一层层的往里面深入学习。最后，在学习完本章内容之后，看看大家能否用一句话给寄存器下一个定义。

6.2 STM32 芯片外观

我们 F407 霸天虎开发板中使用的芯片是 144pin 的 STM32F407ZGT6，具体见图 6-1。这个就是我们接下来要学习的 STM32，它将带领我们进入嵌入式的殿堂。

芯片正面是丝印，ARM 应该是表示该芯片使用的是 ARM 的内核，STM32F429IGT6 是芯片型号，后面的字应该是跟生产批次相关，最下面的是 ST 的 LOGO。

芯片四周是引脚，左下角的小圆点表示 1 脚，然后从 1 脚起按照逆时针的顺序排列（所有芯片的引脚顺序都是逆时针排列的）。开发板中把芯片的引脚引出来，连接到各种传感器上，然后在 STM32 上编程（实际就是通过程序控制这些引脚输出高电平或者低电平）来控制各种传感器工作，通过做实验的方式来学习 STM32 芯片的各个资源。开发板是一种评估板，板载资源非常丰富，引脚复用比较多，力求在一个板子上验证芯片的全部功能。

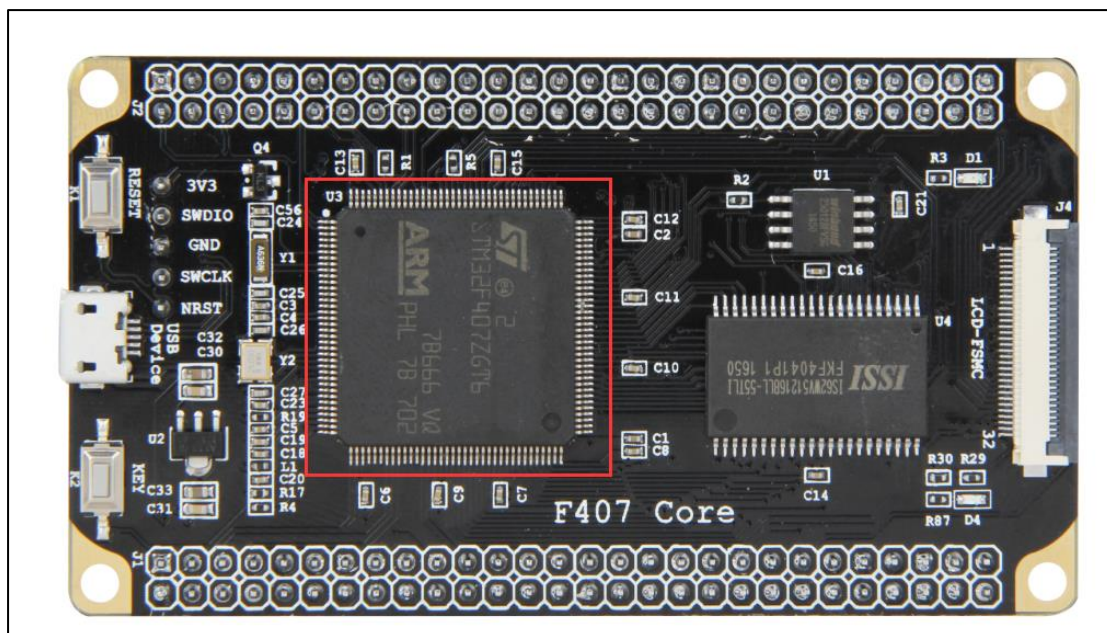


图 6-1 STM32F407ZGT6 实物图(图中方框部分的芯片)

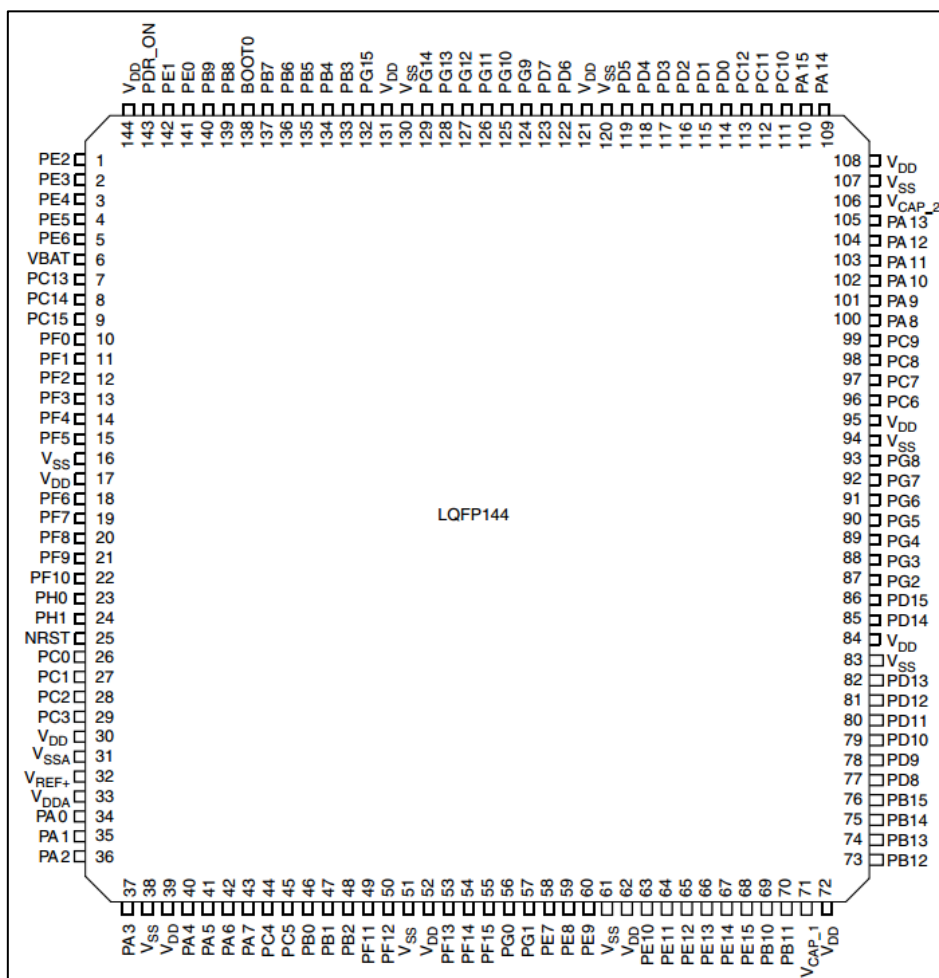


图 6-2 STM32F407ZGT6 正面引脚图

6.3 芯片里面有什么

我们看到的 STM32 芯片已经是已经封装好的成品，主要由内核和片上外设组成。若与电脑类比，内核与外设就如同电脑上的 CPU 与主板、内存、显卡、硬盘的关系。

STM32F407 采用的是 Cortex-M4 内核，内核即 CPU，由 ARM 公司设计。ARM 公司并不生产芯片，而是出售其芯片技术授权。芯片生产厂商(SOC)如 ST、TI、Freescall，负责在内核之外设计部件并生产整个芯片，这些内核之外的部件被称为核外外设或片上外设。如 GPIO、USART（串口）、I2C、SPI 等都叫做片上外设。具体见图 6-3。

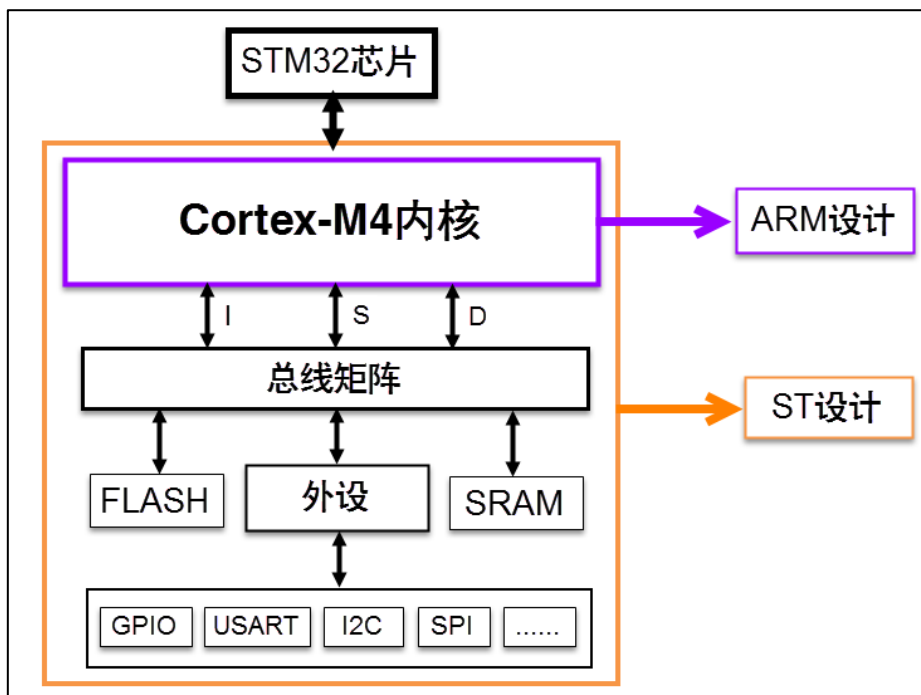


图 6-3 STM32 芯片架构简图

芯片内核和外设之间通过各种总线连接，其中主控总线有 8 条，被控总线有 7 条，具体见图 6-4。主控总线通过一个总线矩阵来连接被控总线，总线矩阵用于主控总线之间的访问仲裁管理，仲裁采用循环调度算法。总线之间交叉的时候如果有个圆圈则表示可以通信，没有圆圈则表示不可以通信。比如 S0：I 总线只有跟 M0、M2 和 M6 这三根被控总线交叉的时候才有圆圈，就表示 S0 只能跟这三根被控总线通信。从功能上来理解，I 总线是指令总线，用来取指，指令指的是编译好的程序指令。我们知道 STM32 有三种启动方式，从 FLASH 启动（包含系统存储器），从内部 SRAM 启动，从外部 RAM 启动，这三种存储器刚好对应的就是 M0、M2 和 M6 这三条总线。

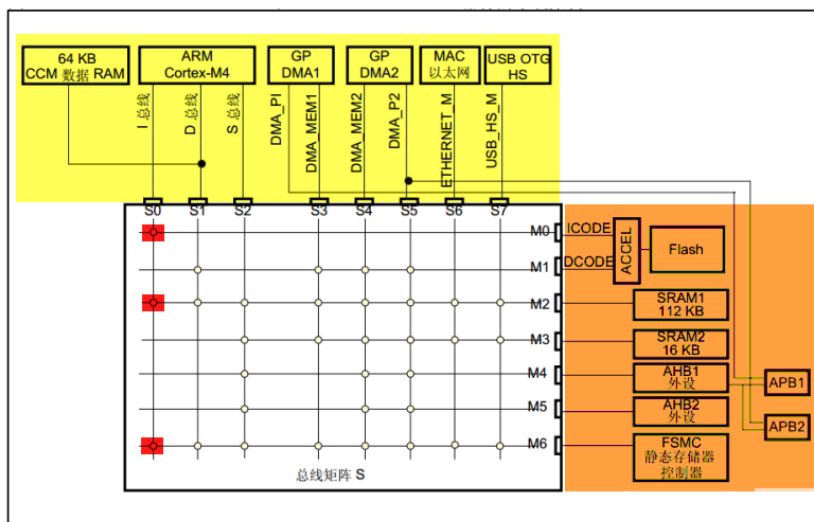


图 6-4 STM32F05xx 和 STM32F407xx 器件的总线接口

6.4 存储器映射

在图 6-4 中，连接被控总线的是 FLASH，RAM 和片上外设，这些功能部件共同排列在一个 4GB 的地址空间内。我们在编程的时候，操作的也正是这些功能部件。

6.4.1 存储器映射

存储器本身不具有地址信息，它的地址是由芯片厂商或用户分配，给存储器分配地址的过程就称为存储器映射，具体见图 6-5。如果给存储器再分配一个地址就叫存储器重映射。

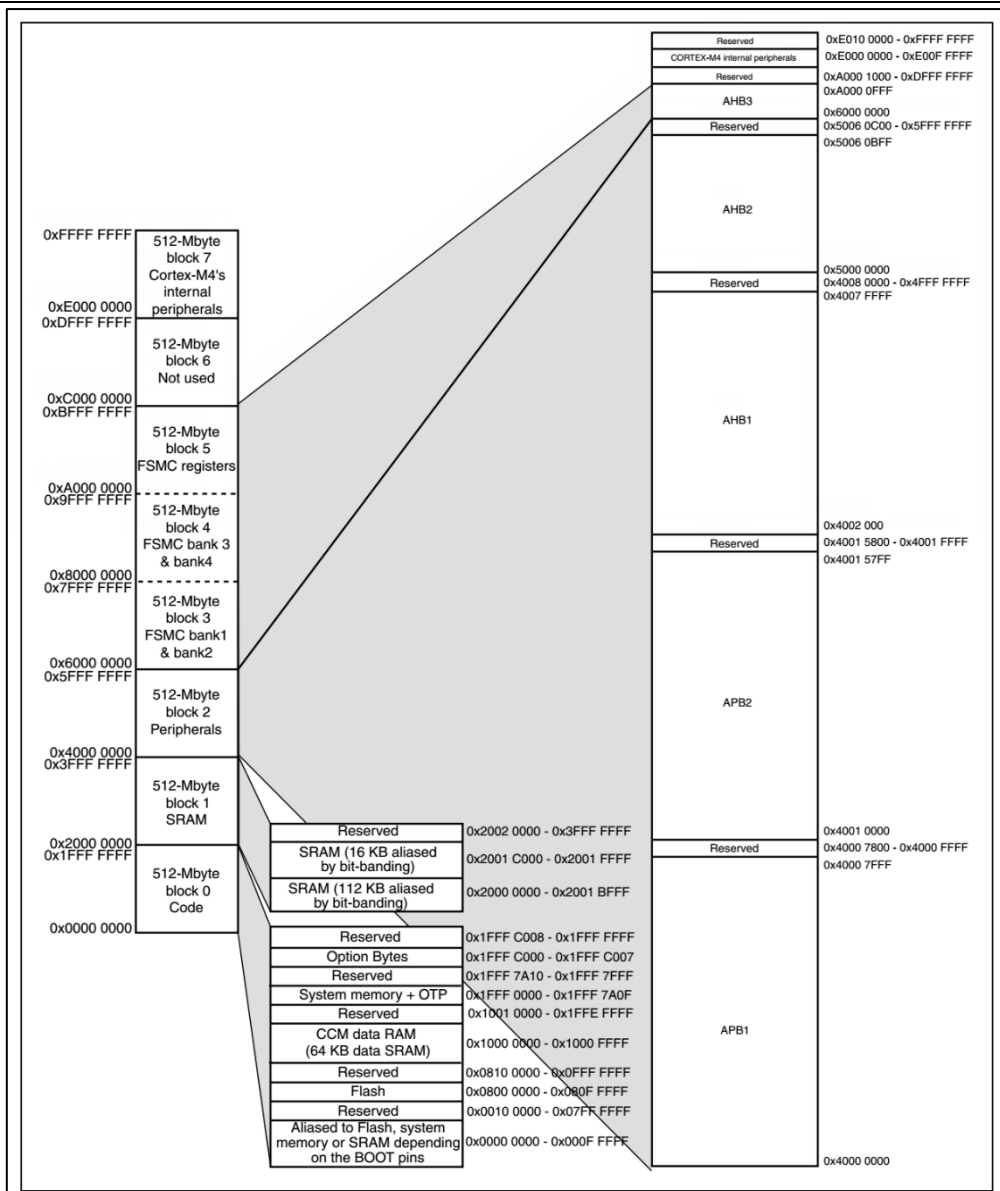


图 6-5 STM32F407 存储器映射

1. 存储器区域功能划分

在这 4GB 的地址空间中，ARM 已经粗线条的平均分成了 8 个块，每块 512MB，每个块也都规定了用途，具体分类见表格 6-1。每个块的大小都有 512MB，显然这是非常大的，芯片厂商在每个块的范围内设计各具特色的外设时并不一定都用得完，都是只用了其中的一部分而已。

表格 6-1 存储器功能分类

序号	用途	地址范围
Block 0	Code	0x0000 0000 ~ 0x1FFF FFFF(512MB)
Block 1	SRAM	0x2000 0000 ~ 0x3FFF FFFF(512MB)
Block 2	片上外设	0x4000 0000 ~ 0x5FFF FFFF(512MB)
Block 3	FSMC 的 bank1 ~ bank2	0x6000 0000 ~ 0x7FFF FFFF(512MB)
Block 4	FSMC 的 bank3 ~ bank4	0x8000 0000 ~ 0x9FFF FFFF(512MB)
Block 5	FSMC 寄存器	0xA000 0000 ~ 0xCFFF FFFF(512MB)

Block 6	没有使用	0xD000 0000 ~ 0xDFFF FFFF(512MB)
Block 7	Cortex-M4 内部外设	0xE000 0000 ~ 0xFFFF FFFF(512MB)

在这 8 个 Block 里面，有 3 个块非常重要，也是我们最关心的三个块。Block0 用来设计成内部 FLASH，Block1 用来设计成内部 RAM，Block2 用来设计成片上的外设，下面我们简单的介绍下这三个 Block 里面的具体区域的功能划分。

存储器 Block0 内部区域功能划分

Block0 主要用于设计片内的 FLASH，F407 系列片内部 FLASH 最大是 1MB，我们使用的 STM32F407ZGT6 的 FLASH 就是 1MB。要在芯片内部集成更大的 FLASH 或者 SRAM 都意味着芯片成本的增加，往往片内集成的 FLASH 都不会太大，ST 能在追求性价比的同时做到 1MB 以上，实乃良心之举。Block0 内部区域的功能划分具体见表格 6-2。

表格 6-2 存储器 Block0 内部区域功能划分

块	用途说明	地址范围
	预留	0x1FFF C008 ~ 0x1FFF FFFF
	选项字节：用于配置读写保护、BOR 级别、软件/硬件看门狗以及器件处于待机或停止模式下的复位。当芯片不小心被锁住之后，我们可以从 RAM 里面启动来修改这部分相应的寄存器位。	0x1FFF C000 ~ 0x1FFF C007
	预留	0x1FFF 7A10 ~ 0x1FFF 7FFF
	系统存储器+OTP：系统存储器里面存的是 ST 出厂时烧写好的 isp 自举程序，用户无法改动。串口下载的时候需要用到这部分程序。OTP 区域，其中 512 个字节只能写一次，用于存储用户数据，额外的 16 个字节用于锁定对应的 OTP 数据块。	0x1FFF 0000 ~ 0x1FFF 7A0F
	预留	0x1001 0000 ~ 0x1FFE FFFF
	CCM 数据 RAM：64K，CPU 直接通过 D 总线读取，不用经过总线矩阵，属于高速的 RAM。	0x1000 0000 ~ 0x1000 FFFF
	预留	0x0810 0000 ~ 0x000F FFFF
	FLASH：我们的程序就放在这里。	0x0800 0000 ~ 0x080F FFFF (1MB)
	预留	0x0010 0000 ~ 0x07FF FFFF
	取决于 BOOT 引脚，为 FLASH、系统存储器、SRAM 的别名。	0x0000 0000 ~ 0x000F FFFF

存储器 Block1 内部区域功能划分

Block1 用于设计片内的 SRAM。F407 内部 SRAM 的大小为 128KB，其中 SRAM1 为 112KB，SRAM2 为 16KB。Block1 内部区域的功能划分具体见表格 6-3。

表格 6-3 存储器 Block1 内部区域功能划分

块	用途说明	地址范围
Block1	预留	0x2002 0000 ~ 0x3FFF FFFF
	SRAM2 16KB	0x2001 C000 ~ 0x2001 FFFF
	SRAM1 112KB	0x2000 0000 ~ 0x2001 BFFF

存储器 Block2 内部区域功能划分

Block2 用于设计片内的外设，根据外设的总线速度不同，Block 被分成了 APB 和 AHB 两部分，其中 APB 又被分为 APB1 和 APB2，AHB 分为 AHB1 和 AHB2，具体见表格 6-4。还有一个 AHB3 包含了 Block3/4/5，AHB3 包含的 3 个 Block 用于扩展外部存储器，如 SRAM，NORFLASH 和 NANDFLASH 等。

表格 6-4 存储器 Block2 内部区域功能划分

块	用途说明	地址范围
Block2	APB1 总线外设	0x4000 0000 ~ 0x4000 7FFF
	预留	0x4000 7800 ~ 0x4000 FFFF
	APB2 总线外设	0x4001 0000 ~ 0x4001 57FF
	预留	0x4001 5800 ~ 0x4001 FFFF
	AHB1 总线外设	0x4002 0000 ~ 0x4007 FFFF
	预留	0x4008 0000 ~ 0x4FFF FFFF
	AHB2 总线外设	0x5000 0000 ~ 0x5006 0BFF
	预留	0x5006 0C00 ~ 0x5FFF FFFF

6.5 寄存器映射

我们知道，存储器本身没有地址，给存储器分配地址的过程叫存储器映射，那什么叫寄存器映射？寄存器到底是什么？

在存储器 Block2 这块区域，设计的是片上外设，它们以四个字节为一个单元，共 32bit，每一个单元对应不同的功能，当我们控制这些单元时就可以驱动外设工作。我们可以找到每个单元的起始地址，然后通过 C 语言指针的操作方式来访问这些单元，如果每次都是通过这种地址的方式来访问，不仅不好记忆还容易出错，这时我们可以根据每个单元功能的不同，以功能为名给这个内存单元取一个别名，这个别名就是我们经常说的寄存器，这个给已经分配好地址的有特定功能的内存单元取别名的过程就叫寄存器映射。

比如，我们找到 GPIOF 端口的输出数据寄存器 ODR 的地址是 0x4002 1414（至于这个地址如何找到可以先跳过，后面我们会有详细的讲解），ODR 寄存器是 32bit，低 16bit 有效，对应着 16 个外部 IO，写 0/1 对应的 IO 则输出低/高电平。现在我们通过 C 语言指针的操作方式，让 GPIOF 的 16 个 IO 都输出高电平，具体见代码 6-1。

代码 6-1 通过绝对地址访问内存单元

```
1 // GPIOF 端口全部输出 高电平
2 *(unsigned int*)(0x4002 1414) = 0xFFFF;
```

0x4002 1414 在我们看来是 GPIOF 端口数据输出寄存器 ODR 的地址，但是在编译器看来，这只是一个普通的变量，是一个立即数，要想让编译器也认为是指针，我们得进行强制类型转换，把它转换成指针，即 (unsigned int *)0x4002 1414，然后再对这个指针进行 * 操作。

刚刚我们说了，通过绝对地址访问内存单元不好记忆且容易出错，我们可以通过寄存器的方式来操作，具体见代码 6-2。

代码 6-2 通过寄存器别名方式访问内存单元

```
1 // GPIOF 端口全部输出 高电平
2 #define GPIOF_BASE          0x40021400
3 #define GPIOF_ODR            (unsigned int*)(GPIOF_BASE+0x14)
```

```
4 *GPIOF_ODR = 0xFF;
```

为了方便操作，我们干脆把指针操作“*”也定义到寄存器别名里面，具体见代码 6-3。

代码 6-3 通过寄存器别名访问内存单元

```
1 // GPIOF 端口全部输出 高电平
2 #define GPIOF_BASE          0x40021400
3 #define GPIOF_ODR            *(unsigned int*)(GPIOF_BASE+0x14)
4 GPIOF_ODR = 0xFF;
```

6.5.1 STM32 的外设地址映射

片上外设区分为四条总线，根据外设速度的不同，不同总线挂载着不同的外设，APB 挂载低速外设，AHB 挂载高速外设。相应总线的最低地址我们称为该总线的基地址，总线基地址也是挂载在该总线上的首个外设的地址。其中 APB1 总线的地址最低，片上外设从这里开始，也叫外设基地址。

1. 总线基地址

表格 6-5 总线基地址

总线名称	总线基地址	相对外设基地址的偏移
APB1	0x4000 0000	0x0
APB2	0x4001 0000	0x0001 0000
AHB1	0x4002 0000	0x0002 0000
AHB2	0x5000 0000	0x1000 0000
AHB3	0x6000 0000	已不属于片上外设

表格 6-5 中的“相对外设基地址偏移”即该总线地址与“片上外设”基地址 0x4000 0000 的差值。关于地址的偏移我们后面还会讲到。

2. 外设基地址

总线上挂载着各种外设，这些外设也有自己的地址范围，特定外设的首个地址称为“XX 外设基地址”，也叫 XX 外设的边界地址。具体有关 STM32F4xx 外设的边界地址请参考《STM32F4xx 参考手册》的 2.3 小节的存储器映射的表 2：STM32F4xx 寄存器边界地址。

这里面我们以 GPIO 这个外设来讲解外设的基地址，具体见表格 6-6。

表格 6-6 外设 GPIO 基地址

外设名称	外设基地址	相对 AHB1 总线的地址偏移
GPIOA	0x4002 0000	0x0
GPIOB	0x4002 0400	0x0000 0400
GPIOC	0x4002 0800	0x0000 0800
GIOD	0x4002 0C00	0x0000 0C00
GPIOE	0x4002 1000	0x0000 1000
GPIOF	0x4002 1400	0x0000 1400
GPIOG	0x4002 1800	0x0000 1800
GPIOH	0x4002 1C00	0x0000 1C00

从表格 6-6 看到，GPIOA 的基址相对于 AHB1 总线的地址偏移为 0，我们应该就可以猜到，AHB1 总线的第一个外设就是 GPIOA。

3. 外设寄存器

在 XX 外设的地址范围内，分布着的就是该外设的寄存器。以 GPIO 外设为例，GPIO 是通用输入输出端口的简称，简单来说就是 STM32 可控制的引脚，基本功能是控制引脚输出高电平或者低电平。最简单的应用就是把 GPIO 的引脚连接到 LED 灯的阴极，LED 灯的阳极接电源，然后通过 STM32 控制该引脚的电平，从而实现控制 LED 灯的亮灭。

GPIO 有很多个寄存器，每一个都有特定的功能。每个寄存器为 32bit，占四个字节，在该外设的基地址上按照顺序排列，寄存器的位置都以相对该外设基地址的偏移地址来描述。这里我们以 GPIOF 端口为例，来说明 GPIO 都有哪些寄存器，具体见表格 6-7。

表格 6-7 GPIOF 端口的 寄存器地址列表

寄存器名称	寄存器地址	相对 GPIOF 基址的偏移
GPIOF_MODER	0x4002 1400	0x00
GPIOF_OTYPER	0x4002 1404	0x04
GPIOF_OSPEEDR	0x4002 1408	0x08
GPIOF_PUPDR	0x4002 140C	0x0C
GPIOF_IDR	0x4002 1410	0x10
GPIOF_ODR	0x4002 1414	0x14
GPIOF_BSRR	0x4002 1418	0x18
GPIOF_LCKR	0x4002 141C	0x1C
GPIOF_AFR1	0x4002 1420	0x20
GPIOF_AFR2	0x4002 1424	0x24

有关外设的寄存器说明可参考《STM32F4xx 参考手册》中具体章节的寄存器描述部分，在编程的时候我们需要反复的查阅外设的寄存器说明。

这里我们以“GPIO 端口置位/复位寄存器”为例，教大家如何理解寄存器的说明，具体见图 6-6。

1

GPIO 端口置位/复位寄存器 (GPIOx_BSRR) (x = A..I)
 GPIO port bit set/reset register

2

偏移地址: 0x18
 复位值: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	— 编号
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0	— 名称
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	— 权限
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	— 编号
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0	— 名称
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	— 权限

4

位 31:16 BRy: 端口 x 复位位 y (Port x reset bit y) (y = 0..15)
 这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回 0x0000。
 0: 不会对相应的 ODRx 位执行任何操作
 1: 对相应的 ODRx 位进行复位
 注意: 如果同时对 BSx 和 BRx 置位，则 BSx 的优先级更高。

位 15:0 BSy: 端口 x 置位位 y (Port x set bit y) (y = 0..15)
 这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回 0x0000。
 0: 不会对相应的 ODRx 位执行任何操作
 1: 对相应的 ODRx 位进行置位

图 6-6 GPIO 端口置位/复位寄存器说明

❑ ①名称

寄存器说明中首先列出了该寄存器中的名称，“(GPIOx_BSRR)(x=A...I)”这段的意思是该寄存器名为“GPIOx_BSRR”其中的“x”可以为 A-I，也就是说这个寄存器说明适用于 GPIOA、GPIOB 至 GPIOI，这些 GPIO 端口都有这样的一个寄存器。

❑ ②偏移地址

偏移地址是指本寄存器相对于这个外设的基地址的偏移。本寄存器的偏移地址是 0x18，从参考手册中我们可以查到 GPIOA 外设的基地址为 0x4002 0000，我们就可以算出 GPIOA 的这个 GPIOA_BSRR 寄存器的地址为：0x4002 0000+0x18；同理，由于 GPIOB 的外设基地址为 0x4002 0400，可算出 GPIOB_BSRR 寄存器的地址为：0x4002 0400+0x18。其他 GPIO 端口以此类推即可。

❑ ③寄存器位表

紧接着的是本寄存器的位表，表中列出它的 0-31 位的名称及权限。表上方的数字为位编号，中间为位名称，最下方为读写权限，其中 w 表示只写，r 表示只读，rw 表示可读写。本寄存器中的位权限都是 w，所以只能写，如果读本寄存器，是无法保证读取到它真正内容的。而有的寄存器位只读，一般是用于表示 STM32 外设的某种工作状态的，由 STM32 硬件自动更改，程序通过读取那些寄存器位来判断外设的工作状态。

❑ ④位功能说明

位功能是寄存器说明中最重要的部分，它详细介绍了寄存器每一个位的功能。例如本寄存器中有两种寄存器位，分别为 BRy 及 BSy，其中的 y 数值可以是 0-15，这里的 0-15 表示端口的引脚号，如 BR0、BS0 用于控制 GPIOx 的第 0 个引脚，若 x 表示 GPIOA，那就是控制 GPIOA 的第 0 引脚，而 BR1、BS1 就是控制 GPIOA 第 1 个引脚。

其中 BRy 引脚的说明是“0：不会对相应的 ODRx 位执行任何操作；1：对相应 ODRx 位进行复位”。这里的“复位”是将该位设置为 0 的意思，而“置位”表示将该位设置为 1；说明中的 ODRx 是另一个寄存器的寄存器位，我们只需要知道 ODRx 位为 1 的时候，对应的引脚 x 输出高电平，为 0 的时候对应的引脚输出低电平即可(感兴趣的读者可以查询该寄存器 GPIOx_ODR 的说明了解)。所以，如果对 BR0 写入“1”的话，那么 GPIOx 的第 0 个引脚就会输出“低电平”，但是对 BR0 写入“0”的话，却不会影响 ODR0 位，所以引脚电平不会改变。要想该引脚输出“高电平”，就需要对“BS0”位写入“1”，寄存器位 BSy 与 BRy 是相反的操作。

6.5.2 C 语言对寄存器的封装

以上所有的关于存储器映射的内容，最终都是为大家更好地理解如何用 C 语言控制读写外设寄存器做准备，此处是本章的重点内容。

1. 封装总线 and 外设基地址

在编程上为了方便理解和记忆，我们把总线基地址和外设基地址都以相应的宏定义起来，总线或者外设都以他们的名字作为宏名，具体见代码 6-4。

代码 6-4 总线和外设基址宏定义

```
1 /* 外设基地址 */
2 #define PERIPH_BASE          ((unsigned int)0x40000000)
3
4 /* 总线基地址 */
5 #define APB1PERIPH_BASE      PERIPH_BASE
6 #define APB2PERIPH_BASE      (PERIPH_BASE + 0x00010000)
7 #define AHB1PERIPH_BASE      (PERIPH_BASE + 0x00020000)
8 #define AHB2PERIPH_BASE      (PERIPH_BASE + 0x10000000)
9
10 /* GPIO 外设基地址 */
11 #define GPIOA_BASE            (AHB1PERIPH_BASE + 0x0000)
12 #define GPIOB_BASE            (AHB1PERIPH_BASE + 0x0400)
13 #define GPIOC_BASE            (AHB1PERIPH_BASE + 0x0800)
14 #define GPIOD_BASE            (AHB1PERIPH_BASE + 0x0C00)
15 #define GPIOE_BASE            (AHB1PERIPH_BASE + 0x1000)
16 #define GPIOF_BASE            (AHB1PERIPH_BASE + 0x1400)
17 #define GPIOG_BASE            (AHB1PERIPH_BASE + 0x1800)
18 #define GPIOH_BASE            (AHB1PERIPH_BASE + 0x1C00)
19
20 /* 寄存器基地址，以 GPIOF 为例 */
21 #define GPIOF_MODER            (GPIOF_BASE+0x00)
22 #define GPIOF_OTYPER            (GPIOF_BASE+0x04)
23 #define GPIOF_OSPEEDR          (GPIOF_BASE+0x08)
24 #define GPIOF_PUPDR            (GPIOF_BASE+0x0C)
25 #define GPIOF_IDR              (GPIOF_BASE+0x10)
26 #define GPIOF_ODR              (GPIOF_BASE+0x14)
27 #define GPIOF_BSRR             (GPIOF_BASE+0x18)
28 #define GPIOF_LCKR             (GPIOF_BASE+0x1C)
29 #define GPIOF_AFR1             (GPIOF_BASE+0x20)
30 #define GPIOF_AFR2             (GPIOF_BASE+0x24)
```

代码 6-4 首先定义了“片上外设”基地址 PERIPH_BASE，接着在 PERIPH_BASE 上加入各个总线的地址偏移，得到 APB1、APB2、AHB1 和 AHB2 总线的基地址 APB1PERIPH_BASE、APB2PERIPH_BASE，AHB1PERIPH_BASE 和 AHB2PERIPH_BASE。然后在 AHB1 总线基地址上加上 GPIO 外设的地址偏移，得到 GPIOA~GPIOH 的外设基地址，最后在外设基地址上加入各寄存器的地址偏移，得到特定寄存器的地址。一旦有了具体地址，就可以用指针进行读写操作，具体见代码 6-5。

代码 6-5 使用指针控制 BSRR 寄存器

```
1 /* 控制 GPIOF 引脚 6 输出低电平 (BSRR 寄存器的 BR6 置 1) */
2 *(unsigned int *)GPIOF_BSRR = (0x01<<(16+6));
3
4 /* 控制 GPIOF 引脚 6 输出高电平 (BSRR 寄存器的 BS6 置 1) */
5 *(unsigned int *)GPIOF_BSRR = 0x01<<6;
6
7 unsigned int temp;
8 /* 控制 GPIOF 端口所有引脚的电平 (读 IDR 寄存器) */
9 temp = *(unsigned int *)GPIOF_IDR;
```

该代码使用 (unsigned int *) 把 GPIOF_BSR 宏的数值强制转换成了地址，然后再用 “*” 号做取指针操作，对该地址的赋值，从而实现了写寄存器的功能。同样，读寄存器也是用取指针操作，把寄存器中的数据取到变量里，从而获取 STM32 外设的状态。

2. 封装寄存器列表

用上面的方法去定义地址，还是稍显繁琐，例如 GPIOA~GPIOH 都各有一组功能相同的寄存器，如 GPIOA_MODER/GPIOB_MODER/GPIOC_MODER 等等，它们只是地址不一样，但却要为每个寄存器都定义它的地址。为了方便地访问寄存器，我们引入 C 语言中的结构体语法对寄存器进行封装，具体见代码 6-6。

代码 6-6 使用结构体对 GPIO 寄存器组的封装

```
1 typedef unsigned int uint32_t; /*无符号 32 位变量*/
2 typedef unsigned short int uint16_t; /*无符号 16 位变量*/
3
4 /* GPIO 寄存器列表 */
5 typedef struct {
6     uint32_t MODER; /*GPIO 模式寄存器 地址偏移: 0x00 */
7     uint32_t OTYPER; /*GPIO 输出类型寄存器 地址偏移: 0x04 */
8     uint32_t OSPEEDR; /*GPIO 输出速度寄存器 地址偏移: 0x08 */
9     uint32_t PUPDR; /*GPIO 上拉/下拉寄存器 地址偏移: 0x0C */
10    uint32_t IDR; /*GPIO 输入数据寄存器 地址偏移: 0x10 */
11    uint32_t ODR; /*GPIO 输出数据寄存器 地址偏移: 0x14 */
12    uint16_t BSRRL; /*GPIO 置位/复位寄存器低 16 位部分 地址偏移: 0x18 */
13    uint16_t BSRRH; /*GPIO 置位/复位寄存器高 16 位部分 地址偏移: 0x1A */
14    uint32_t LCKR; /*GPIO 配置锁定寄存器 地址偏移: 0x1C */
15    uint32_t AFR[2]; /*GPIO 复用功能配置寄存器 地址偏移: 0x20-0x24 */
16 } GPIO_TypeDef;
```

这段代码用 typedef 关键字声明了名为 GPIO_TypeDef 的结构体类型，结构体内有 8 个成员变量，变量名正好对应寄存器的名字。C 语言的语法规则规定，结构体内变量的存储空间是连续的，其中 32 位的变量占用 4 个字节，16 位的变量占用 2 个字节，具体见图 6-7。

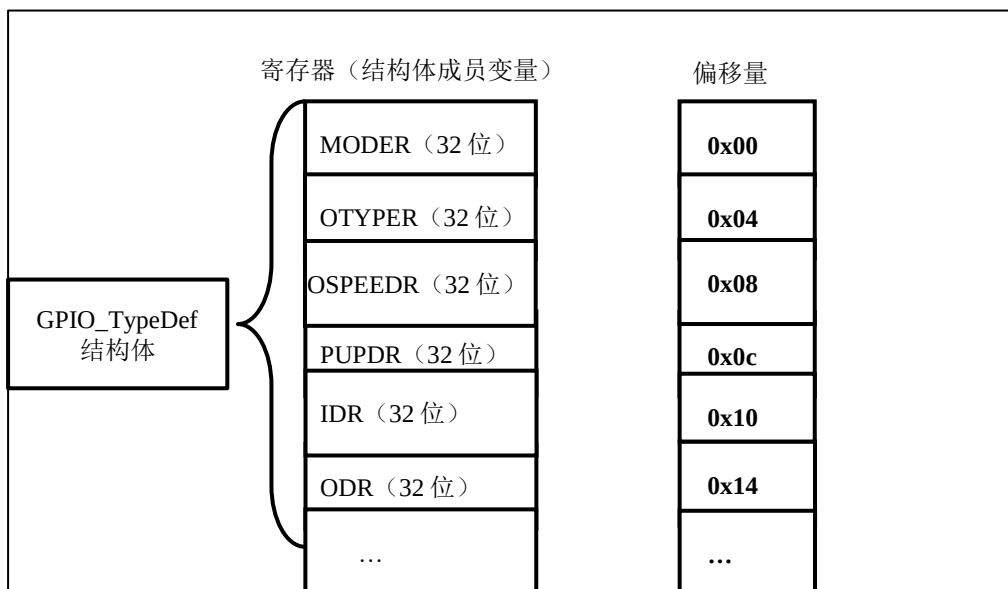


图 6-7 GPIO_TypeDef 结构体成员的地址偏移

也就是说，假如我们定义一个 GPIO_TypeDef 类型的结构体，且结构体的首地址为 0x4002 1400（这也是第一个成员变量 MODER 的地址），那么结构体中第二个成员变量 OTYPER 的地址即为 0x4002 1400 + 0x04，加上的这个 0x04，正是代表 MODER 所占用的 4 个字节地址的偏移量，其它成员变量相对于结构体首地址的偏移，在上述代码右侧注释已给出，其中的 BSRR 寄存器分成了低 16 位 BSRR_L 和高 16 位 BSRR_H，BSRR_L 置 1 引脚输出高电平，BSRR_H 置 1 引脚输出低电平，这里分开只是为了方便操作。

这样的地址偏移与 STM32 GPIO 外设定义的寄存器地址偏移一一对应，只要给结构体设置好首地址，就能把结构体内成员的地址确定下来，然后就能以结构体的形式访问寄存器了，具体见代码 6-7。

代码 6-7 通过结构体指针访问寄存器

```
1 GPIO_TypeDef * GPIOx;           //定义一个 GPIO_TypeDef 型结构体指针 GPIOx
2 GPIOx = GPIOF_BASE;            //把指针地址设置为宏 GPIOF_BASE 地址
3 GPIOx->BSRR = 0xFFFF;          //通过指针访问并修改 GPIOF_BSRR 寄存器
4 GPIOx->MODER = 0xFFFFFFFF;      //修改 GPIOF_MODER 寄存器
5 GPIOx->OTYPER = 0xFFFFFFFF;     //修改 GPIOF_OTYPER 寄存器
6
7 uint32_t temp;
8 temp = GPIOx->IDR;              //读取 GPIOF_IDR 寄存器的值到变量 temp 中
```

这段代码先用 GPIO_TypeDef 类型定义一个结构体指针 GPIOx，并让指针指向地址 GPIOF_BASE(0x4002 1400)，使地址确定下来，然后根据 C 语言访问结构体的语法，用 GPIOx->BSRR、GPIOx->MODER 及 GPIOx->IDR 等方式读写寄存器。

最后，我们更进一步，直接使用宏来定义好 GPIO_TypeDef 类型的指针，而且指针指向各个 GPIO 端口的首地址，使用时我们直接用该宏访问寄存器即可，具体代码 6-8。

代码 6-8 定义好 GPIO 端口首地址指针

```
1 /*使用 GPIO_TypeDef 把地址强制转换成指针*/
2 #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
3 #define GPIOB ((GPIO_TypeDef *) GPIOB_BASE)
4 #define GPIOC ((GPIO_TypeDef *) GPIOC_BASE)
5 #define GPIOD ((GPIO_TypeDef *) GPIOD_BASE)
6 #define GPIOE ((GPIO_TypeDef *) GPIOE_BASE)
7 #define GPIOF ((GPIO_TypeDef *) GPIOF_BASE)
8 #define GPIOG ((GPIO_TypeDef *) GPIOG_BASE)
9 #define GPIOH ((GPIO_TypeDef *) GPIOH_BASE)
10
11
12
13 /*使用定义好的宏直接访问*/
14 /*访问 GPIOF 端口的寄存器*/
15 GPIOF->BSRR = 0xFFFF;          //通过指针访问并修改 GPIOF_BSRR 寄存器
16 GPIOF->MODER = 0xFFFFFFFF;     //修改 GPIOF_MODER 寄存器
17 GPIOF->OTYPER = 0xFFFFFFFF;    //修改 GPIOF_OTYPER 寄存器
18
19 uint32_t temp;
20 temp = GPIOF->IDR;              //读取 GPIOF_IDR 寄存器的值到变量 temp 中
21
22 /*访问 GPIOA 端口的寄存器*/
23 GPIOA->BSRR = 0xFFFF;          //通过指针访问并修改 GPIOA_BSRR 寄存器
24 GPIOA->MODER = 0xFFFFFFFF;     //修改 GPIOA_MODER 寄存器
25 GPIOA->OTYPER = 0xFFFFFFFF;    //修改 GPIOA_OTYPER 寄存器
26
27 uint32_t temp;
```



```
28 temp = GPIOA->IDR; //读取 GPIOA_IDR 寄存器的值到变量 temp 中
```

这里我们仅是以 GPIO 这个外设为例，给大家讲解了 C 语言对寄存器的封装。以此类推，其他外设也同样可以用这种方法来封装。好消息是，这部分工作都由固件库帮我们完成了，这里我们只是分析了下这个封装的过程，让大家知其然，也只其所以然。

6.5.3 修改寄存器的位操作方法

使用 C 语言对寄存器赋值时，我们常常要求只修改该寄存器的某几位的值，且其它的寄存器位不变，这个时候我们就需要用到 C 语言的位操作方法了。

1. 把变量的某位清零

此处我们以变量 a 代表寄存器，并假设寄存器中本来已有数值，此时我们需要把变量 a 的某一位清零，且其它位不变，方法见代码清单 6-1。

代码清单 6-1 对某位清零

```
1 //定义一个变量 a = 1001 1111 b (二进制数)
2 unsigned char a = 0x9f;
3
4 //对 bit2 清零
5
6 a &= ~(1<<2);
7
8 //括号中的 1 左移两位，(1<<2)得二进制数：0000 0100 b
9 //按位取反，~(1<<2)得 1111 1011 b
10 //假如 a 中原来的值为二进制数： a = 1001 1111 b
11 //所得的数与 a 作“位与&”运算，a = (1001 1111 b)&(1111 1011 b)，
12 //经过运算后，a 的值 a=1001 1011 b
13 // a 的 bit2 位被清零，而其它位不变。
```

2. 把变量的某几个连续位清零

由于寄存器中有时会有连续几个寄存器位用于控制某个功能，现假设我们需要把寄存器的某几个连续位清零，且其它位不变，方法见代码清单 6-2。

代码清单 6-2 对某几个连续位清零

```
1
2 //若把 a 中的二进制位分成 2 个一组
3 //即 bit0、bit1 为第 0 组，bit2、bit3 为第 1 组，
4 // bit4、bit5 为第 2 组，bit6、bit7 为第 3 组
5 //要对第 1 组的 bit2、bit3 清零
6
7 a &= ~(3<<2*1);
8
9 //括号中的 3 左移两位，(3<<2*1)得二进制数：0000 1100 b
10 //按位取反，~(3<<2*1)得 1111 0011 b
11 //假如 a 中原来的值为二进制数： a = 1001 1111 b
12 //所得的数与 a 作“位与&”运算，a = (1001 1111 b)&(1111 0011 b)，
13 //经过运算后，a 的值 a=1001 0011 b
14 // a 的第 1 组的 bit2、bit3 被清零，而其它位不变。
15
16 //上述(~(3<<2*1))中的(1)即为组编号；如清零第 3 组 bit6、bit7 此处应为 3
17 //括号中的(2)为每组的位数，每组有 2 个二进制位；若分成 4 个一组，此处即为 4
18 //括号中的(3)是组内所有位都为 1 时的值；若分成 4 个一组，此处即为二进制数“1111 b”
```



```
19
20 //例如对第 2 组 bit4、bit5 清零
21 a &= ~(3<<2*2);
```

3. 对变量的某几位进行赋值。

寄存器位经过上面的清零操作后，接下来就可以方便地对某几位写入所需要的数值了，且其它位不变，方法见代码清单 6-3，这时候写入的数值一般就是需要设置寄存器的位参数。

代码清单 6-3 对某几位进行赋值

```
1 //a = 1000 0011 b
2 //此时对清零后的第 2 组 bit4、bit5 设置成二进制数"01 b "
3
4 a |= (1<<2*2);
5 //a = 1001 0011 b, 成功设置了第 2 组的值，其它组不变
```

4. 对变量的某位取反

某些情况下，我们需要对寄存器的某个位进行取反操作，即 1 变 0，0 变 1，这可以直接用如下操作，其它位不变，见代码清单 6-4。

代码清单 6-4 对某位进行取反操作

```
1 //a = 1001 0011 b
2 //把 bit6 取反，其它位不变
3
4 a ^= (1<<6);
5 //a = 1101 0011 b
```

关于修改寄存器位的这些操作，在下一章中有应用实例代码，可配合阅读。